

Part 01 - Questions

Q1: What is the default value assigned to array elements in C#?

Answer: The default value for numeric array elements is 0 (or 0.0 for floating points, false for booleans, and null for reference types like strings).

Q2: What is the difference between `Array.Clone()` and `Array.Copy()`?

Answer: `Array.Clone()` creates a shallow copy of the entire array and returns a new object that needs casting. `Array.Copy()` copies a specified range of elements from one existing array into another pre-allocated destination array.

Q3: What is the difference between `GetLength()` and `Length` for multi-dimensional arrays?

Answer: `Length` returns the total number of elements across all dimensions combined. `GetLength(dimension)` returns the number of elements in a specific dimension (e.g., `GetLength(0)` for rows, `GetLength(1)` for columns).

Q4: What is the difference between `Array.Copy()` and `Array.ConstrainedCopy()`?

Answer: `Array.Copy()` stops at the point of failure if an exception occurs during copying, leaving partial modifications in the destination array. `Array.ConstrainedCopy()` guarantees atomic behavior: if a type mismatch occurs during the process, it rolls back entirely without modifying the target array.

Q5: Why is `foreach` preferred for read-only operations on arrays?

Answer: `foreach` is preferred because it prevents accidental modification of array elements, hides index tracking logic, improves readability, and completely eliminates `IndexOutOfRangeException` errors.

Q6: Why is input validation important when working with user inputs?

Answer: Input validation prevents application crashes, protects against bad data processing, secures the system against malicious inputs, and ensures a smooth user experience.

Q7: How can you format the output of a 2D array for better readability?

Answer: By using nested loops combined with tab formatting (`\t`) between elements and printing a new line (`Console.WriteLine()`) after each row to render data in a clean grid/matrix structure.

Q8: When should you prefer a `switch` statement over `if-else`?

Answer: You should prefer `switch` when comparing a single variable against multiple discrete static values (like numbers or enums) because it is cleaner to read and optimized by the compiler using jump tables.

Q9: What is the time complexity of Array.Sort()?

Answer: The time complexity is $O(N \log N)$ on average because C# uses Introsort (a hybrid of QuickSort, HeapSort, and InsertionSort).

Q10: Which loop (for or foreach) is more efficient for calculating the sum of an array, and why?

Answer: For single-dimensional arrays, `for` is slightly more efficient because the compiler optimizes out index boundary checks during iteration. However, the performance gap is negligible in most real-world scenarios.

Part 02 - Questions

Q3: What happens if the user enters a value outside the range of 1 to 7?

Answer: `Enum.Parse` will still successfully convert the integer into the enum type without throwing an exception, but it will store the underlying numeric value instead of matching a named enum constant.

Part 03 (Bonus) - Questions

Q5: What's the default size of Stack and Heap, and what are the considerations?

Answer:

Stack: Has a fixed default size of 1 MB per thread. It handles quick short-lived allocations, and exceeding it causes a `StackOverflowException`.

Heap: Has a dynamic size limited only by available system RAM and process address space. It handles reference types and is managed automatically by the Garbage Collector.

Q6: What is time complexity?

Answer: Time complexity is a mathematical representation (Big-O notation) used to measure how the execution time of an algorithm grows as the size of the input data (N) increases.